**Patrones de software**

**Implementación de patrones v1**

**Docente: ELIECER MONTERO OJEDA**

**Ángel David Reyes Moreno**

**UNIDADES TECNOLÓGICAS DE SANTANDER**
**Ingenieria de Sistemas**
**Bucaramanga, 2026**

**Patrón 1: Singleton**

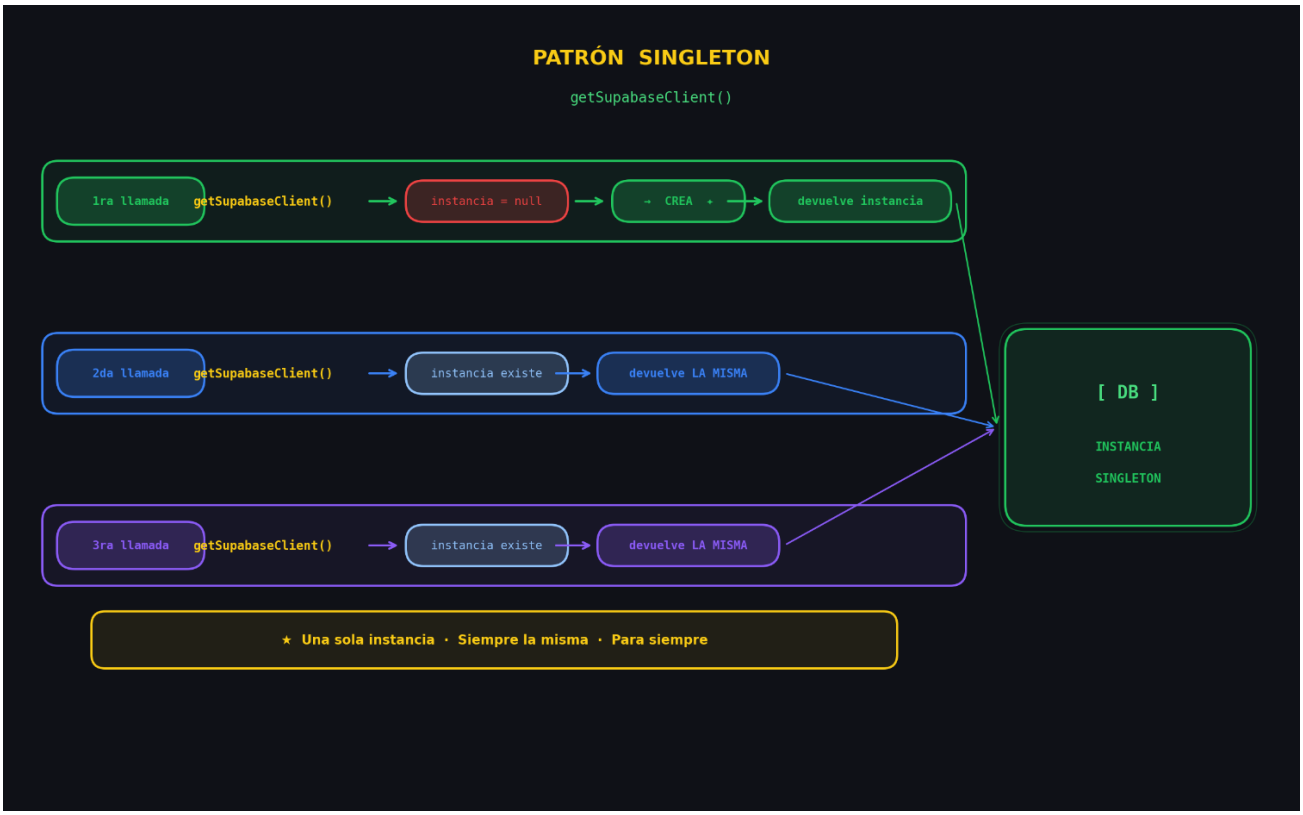
¿Qué problema resuelve? Garantiza que solo exista una única instancia de un objeto durante toda la vida de la aplicación. Útil para configuración y conexiones a bases de datos.

```
export class ConfigManager {
  // 1. Variable estática que guarda LA única instancia
  private static instance: ConfigManager | null = null;

  // 2. Constructor PRIVADO → nadie puede hacer "new ConfigManager()"
  private constructor() {
    this.config = { /* lee las variables de entorno */ }
  }

  // 3. Punto de acceso global → siempre devuelve la misma instancia
  public static getInstance(): ConfigManager {
    if (!ConfigManager.instance) {
      ConfigManager.instance = new ConfigManager(); // solo se crea UNA vez
    }
    return ConfigManager.instance; // las demás veces, devuelve la existente
  }
}
```

¿Dónde se usa? El cliente de Supabase lo llama al iniciarse:

```
let supabaseInstance: SupabaseClient | null = null;

export function getSupabaseClient(): SupabaseClient {
  if (supabaseInstance) return supabaseInstance; // ya existe → devuelve la misma

  const config = ConfigManager.getInstance(); // ← usa el otro Singleton
  supabaseInstance = createClient(config.get("supabaseUrl"), ...);
  return supabaseInstance;
}
```

Diagrama Uml


PATRÓN SINGLETON
getSupabaseClient()
1ra llamada
getSupabaseClient()
instancia = null
→ CREA ✦
devuelve instancia
2da llamada
getSupabaseClient()
instancia existe
devuelve LA MISMA
3ra llamada
getSupabaseClient()
instancia existe
devuelve LA MISMA
[ DB ]
INSTANCIA
SINGLETON
★ Una sola instancia · Siempre la misma · Para siempre

**Patrón 2: Factory Method**

¿Qué problema resuelve? Delegar la **creación de objetos** a subclases, sin que el código cliente sepa qué clase concreta se está creando. Perfecto para múltiples tipos de dispositivos IoT.

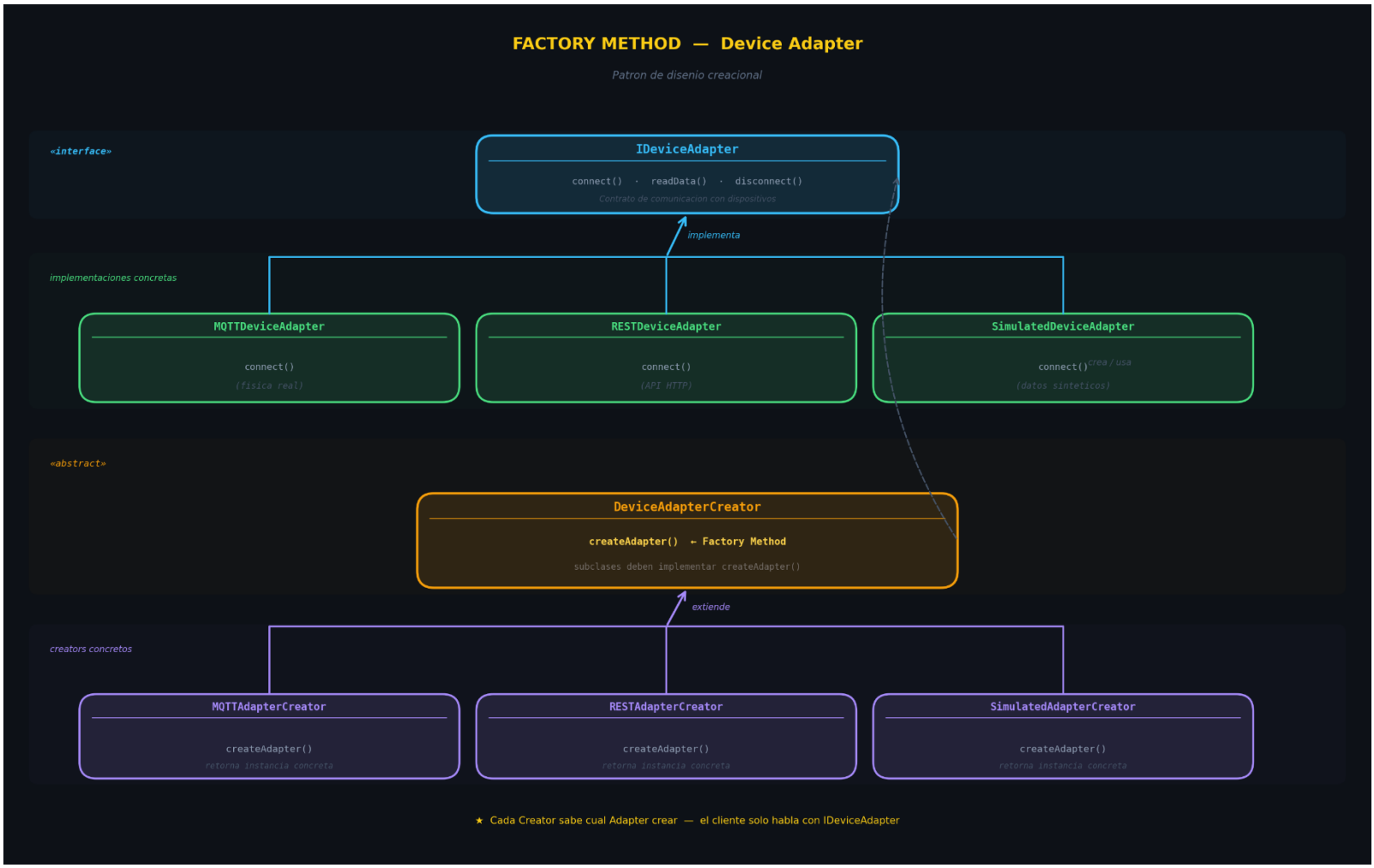
```
// Creator abstracto define el "molde"
abstract class DeviceAdapterCreator {
  abstract createAdapter(id: string, type: DeviceType): IDeviceAdapter; // ← Factory Method
}

// Cada subclase decide QUÉ adaptador concreto crear
class SimulatedAdapterCreator extends DeviceAdapterCreator {
  createAdapter(id: string, type: DeviceType): IDeviceAdapter {
    return new SimulatedDeviceAdapter(id, type); // concreta
  }
}

// Punto de entrada público — elige el Creator correcto
export class DeviceAdapterFactory {
  static async create(params): Promise<IDeviceAdapter> {
    switch (params.protocol) {
      case "MQTT":      return new MQTTAdapterCreator().getAndConnect(...);
      case "REST":      return new RESTAdapterCreator().getAndConnect(...);
      case "SIMULATED": return new SimulatedAdapterCreator().getAndConnect(...);
    }
  }
}
```

¿Dónde se usa en la UI?

La UI solo llama a la fábrica. De momento se esta instanciando un protocolo de simulación si en un futuro se implementa un protocolo en tiempo reaL solo hay que crear ese protocolo ejemplo: ZIGBEE, solo  ZigbeeDeviceAdapter y ZigbeeAdapterCreator — el código de la UI no cambia para nada seguirá llamando a la fabrica .

```
// La página NO sabe qué adaptador concreto se crea
const adapter = await DeviceAdapterFactory.create({
  deviceId: "dev-001",
  deviceType: "SOLAR_PANEL",
  protocol: "SIMULATED",  // ← aquí solo cambia el protocolo
});
const reading = await adapter.readData(); // misma interfaz siempre
```

Diagrama Uml


FACTORY METHOD — Device Adapter
Patron de disenio creacional
«interface»
IDeviceAdapter
connect() · readData() · disconnect()
Contrato de comunicacion con dispositivos
implementa
implementaciones concretas
MQTTDeviceAdapter
connect()
(fisica real)
RESTDeviceAdapter
connect()
(API HTTP)
SimulatedDeviceAdapter
connect()
crea / usa
(datos sinteticos)
«abstract»
DeviceAdapterCreator
createAdapter() ← Factory Method
subclases deben implementar createAdapter()
extiende
creators concretos
MQTTAdapterCreator
createAdapter()
retorna instancia concreta
RESTAdapterCreator
createAdapter()
retorna instancia concreta
SimulatedAdapterCreator
createAdapter()
retorna instancia concreta
★ Cada Creator sabe cual Adapter crear — el cliente solo habla con IDeviceAdapter